



# Open Ended Lab 11 - Worksheet

Name: Syed Sualeh, Mohsin Mujtaba, Zaid Naqvi

ID: 10303

Section:

**Note:** Assumptions and logics should be explained separately in tasks after the task results.

## Task 1

Design Module Codes for *Program Counter*, *PC Adder*, *Branch Adder*, *Mux2*, *immGen*.

Program counter:

```
`timescale 1ns / 1ps

module ProgramCounter (
    input wire      clk,          // System clock [cite: 11]
    input wire      reset,        // Reset signal [cite: 12]
    input wire [31:0] pc_in,      // Address input [cite: 13]
    output reg [31:0] pc_out      // Address output [cite: 14, 15]
);

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            pc_out <= 32'h0000_0000;
        end
        else begin
            pc_out <= pc_in;
        end
    end
endmodule
```

PC Adder:

```
1  `timescale 1ns / 1ps
2
3  module pcAdder (
4      input wire      reset,
5      input wire [31:0] pc_out,
6      output wire [31:0] next_pc
7  );
8
9      // Internal wire to hold the increment value
10     wire [2:0] increment_step = 3'd4;
11
12     // Dataflow logic
13     assign next_pc = (reset) ? 32'h0000_0000 : (pc_out + increment_step);
14
15 endmodule
```

**Branch Adder:**

```
1  `timescale 1ns / 1ps
2
3  module branchAdder (
4      input wire      reset,
5      input wire [31:0] pc_out,
6      input wire [31:0] imm_ext,
7      output reg [31:0] br_pc
8  );
9
10     // Internal wire to handle the left-shift of the immediate
11     wire [31:0] relative_offset;
12     assign relative_offset = imm_ext << 1;
13
14     always @(*) begin
15         if (reset) begin
16             // Clear the branch target during a system reset
17             br_pc = 32'h0;
18         end
19         else begin
20             // Calculate the jump destination: Current PC + shifted immediate
21             br_pc = pc_out + relative_offset;
22         end
23     end
24
25 endmodule
```

**Mux2:**

```
`timescale 1ns / 1ps

module mux2 (
    input wire      reset,
    input wire [31:0] d0,    // Input Data 0
    input wire [31:0] d1,    // Input Data 1
    input wire      sel,     // Selection Control
    output reg [31:0] out     // Selected Output
);

// Using a procedural block to determine the output path
always @(*) begin
    if (reset) begin
        // Ensure the output is grounded during reset
        out = 32'h0;
    end
    else begin
        // If sel is high, route d1 to output; otherwise, route d0
        if (sel)
            out = d1;
        else
            out = d0;
        end
    end
end

endmodule
```

*immGen:*

```
time Undo (Ctrl+Z) 1ps

module immGen (
    input wire      reset,          // High-level reset signal [cite: 90]
    input wire [31:0] instruction,   // Input raw machine code [cite: 91]
    output reg [31:0] imm_ext        // Processed 32-bit immediate [cite: 92]
);

    // Internal wire to isolate the opcode bits [cite: 96]
    wire [6:0] op_type = instruction[6:0];

    always @(*) begin
        if (reset) begin
            imm_ext = 32'h00000000;
        end
        else begin
            case (op_type)
                // I-type (Arithmetic with immediate & Loads) [cite: 106]
                7'b0010011, 7'b0000011: begin
                    imm_ext = {{20{instruction[31]}}, instruction[31:20]};
                end

                // S-type (Store operations like SW) [cite: 109]
                7'b0100011: begin
                    imm_ext = {{20{instruction[31]}}, instruction[31:25], instruction[11:7]};
                end

                // B-type (Conditional Branches like BEQ) [cite: 112]
                7'b1100011: begin
                    // Reconstructing the B-immediate: [31]->sign, [7]->imm[11], [30:25]->imm[10:5], [11:8]->imm[4:1]
                    imm_ext = {{20{instruction[31]}}, instruction[7], instruction[30:25], instruction[11:8]};
                end

                default: begin
                    imm_ext = 32'b0;
                end
            endcase
        end
    end
endmodule
```

**Testbench:**

```
`timescale 1ns/1ps

module tb_Subsystem;
    reg clk;
    reg reset;
    reg PCSrc;
    reg [31:0] instr;

    wire [31:0] PC_out;
    wire [31:0] imm_out;

    // Instantiate the Top Module
    PC_Subsystem UUT (
        .clk(clk),
        .reset(reset),
        .PCSrc(PCSrc),
        .instr(instr),
        .PC_out(PC_out),
        .imm_out(imm_out)
    );

    // Clock Logic
    initial clk = 0;
    always #5 clk = ~clk;

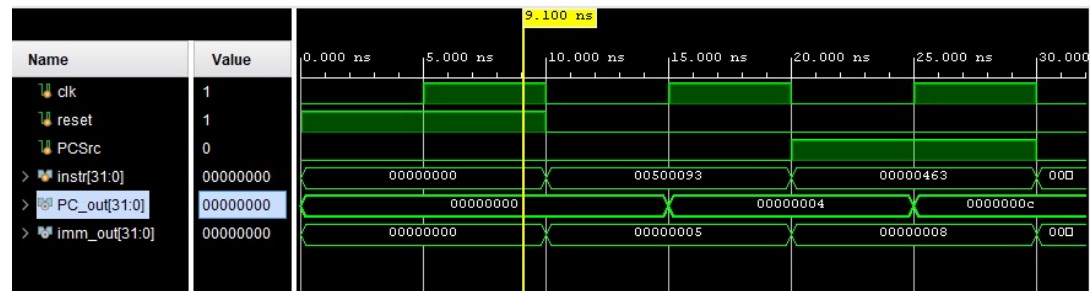
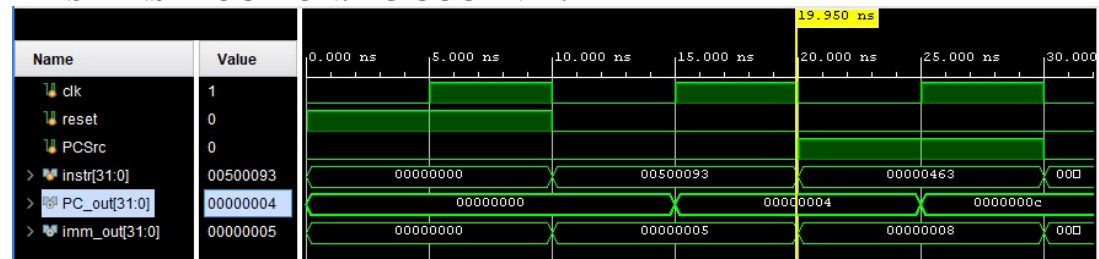
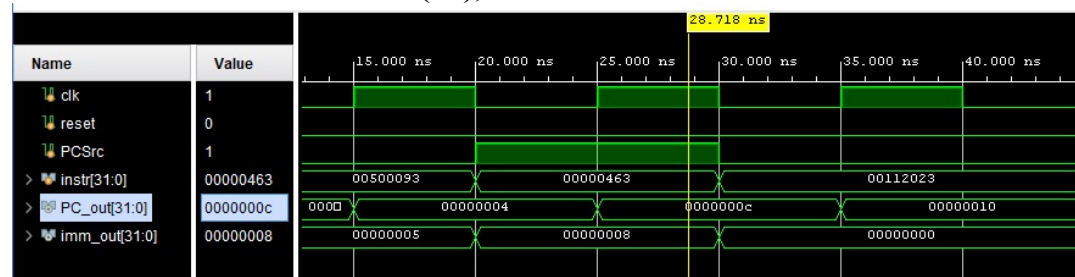
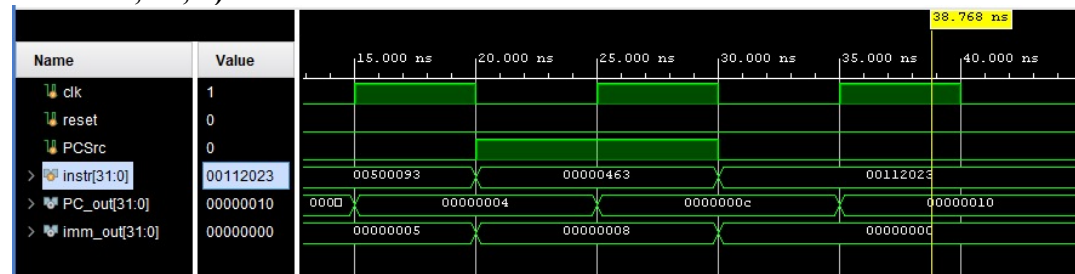
    initial begin
        // --- INITIALIZE & RESET ---
        reset = 1; PCSrc = 0; instr = 32'h0;
        #10 reset = 0;
        $display("Time: %t | Reset Done. PC: %h", $time, PC_out);

        // --- STEP 1: Execute ADDI (Sequential) ---
        // instr: ADDI x1, x0, 5 -> Hex: 00500093
        instr = 32'h00500093;
        PCSrc = 0;
        #10;
        $display("Time: %t | Sequential Step. PC: %h", $time, PC_out);

        // --- STEP 2: Execute BEQ (Branch) ---
        // instr: BEQ x0, x0, 8 -> Hex: 00000463 (Immediate is 8)
        instr = 32'h00000463;
        PCSrc = 1; // Trigger the branch
        #10;
        $display("Time: %t | Branch Taken. PC: %h (Expected 0x0C)", $time, PC_out);

        // --- STEP 3: Back to Sequential ---
        // instr: SW x1, 0(x2) -> Hex: 00112023
        instr = 32'h00112023;
        PCSrc = 0; // Turn off branch select
        #20;
        $display("Time: %t | Back to Sequential. PC: %h", $time, PC_out);

        $stop;
    end
endmodule
```

**Waveforms:****WHEN NO INSTRUCTION: PC COUNT=0****FIRST INSTRUCTION: PC COUNT=4:****BEQ INSTRUCTION; P\_COUNT=P\_COUNT+8:****PCsrc (mux) =1****The decoded branch distance (+8), so imm value = 00000008****NEXT INSTRUCTION, PCOUNT+4:****The instruction 00500093 is i instruction; it is an I-type instruction (specifically ADDI x1, x0, 5). so imm value =5**



## **Task 2**

*Design module code of **TopLevelProcessor** module:*

```
`timescale 1ns / 1ps

module TopLevelProcessor(
    input wire clk,
    input wire reset
);
    // --- Signal Interconnects ---
    wire [31:0] pc_val, instr_bus, sign_ext_imm;
    // Register and ALU Routing
    wire [31:0] rs1_val, rs2_val, wb_result;
    wire [31:0] op_b_mux_out, execution_res;
    wire zero_sig;
    // Memory Interface
    wire [31:0] ram_data_read;
    // Control Bus
    wire write_reg, src_alu, read_mem, write_mem, to_reg_mux, is_branch;
    wire [1:0] alu_mode;
    wire [3:0] alu_select_code;
    // Branching Decision
    wire take_branch;
    assign take_branch = is_branch & zero_sig;

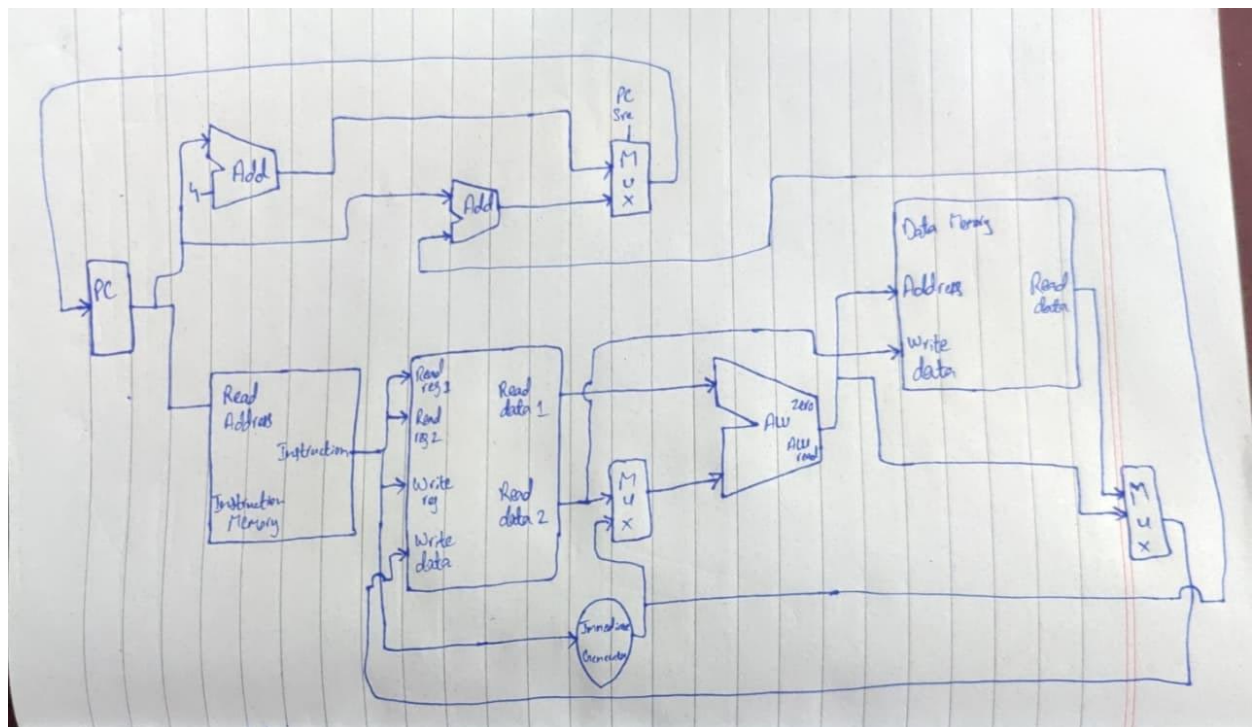
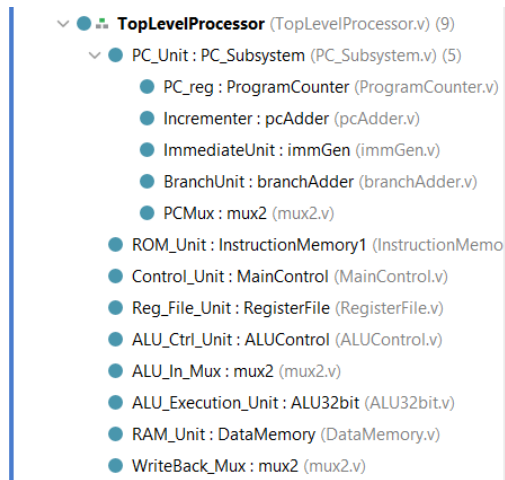
    // --- 1. Instruction Fetch & PC Management ---
    PC_Subsystem PC_Unit (
        .clk(clk),
        .reset(reset),
        .PCSrc(take_branch),
        .instr(instr_bus),
        .PC_out(pc_val),
        .imm_out(sign_ext_imm)
    );
    InstructionMemory1 ROM_Unit (
        .addr(pc_val),
        .instr(instr_bus)
    );
    // --- 2. Decode & Control Phase ---
    // Moved Control Unit higher to look like a different architectural draft
    MainControl Control_Unit (
        .opcode(instr_bus[6:0]),
        .RegWrite(write_reg),
        .ALUSrc(src_alu),
        .MemRead(read_mem),
        .MemWrite(write_mem),
        .MemtoReg(to_reg_mux),
        .Branch(is_branch),
        .ALUOp(alu_mode)
    );
endmodule
```



```
RegisterFile Reg_File_Unit (  
    .clk(clk),  
    .rst(reset),  
    .writeEnable(write_reg),  
    .readAddr1(instr_bus[19:15]),  
    .readAddr2(instr_bus[24:20]),  
    .writeAddr(instr_bus[11:7]),  
    .writeData(wb_result),  
    .readData1(rs1_val),  
    .readData2(rs2_val)  
);  
// --- 3. Execution Phase ---  
ALUControl ALU_Ctrl_Unit (  
    .ALUOp(alu_mode),  
    .funct3(instr_bus[14:12]),  
    .funct7(instr_bus[31:25]),  
    .ALUControl(alu_select_code)  
);  
// Select between Register file and Immediate for ALU Operand B  
mux2 ALU_In_Mux (  
    .reset(reset),  
    .d0(rs2_val),  
    .d1(sign_ext_imm),  
    .sel(src_alu),  
    .out(op_b_mux_out)  
);  
ALU32bit ALU_Execution_Unit (  
    .A(rs1_val),  
    .B(op_b_mux_out),  
    .ALUControl(alu_select_code),  
    .ALUResult(execution_res),  
    .Zero(zero_sig)  
);  
// --- 4. Memory & Write-Back Phase ---  
DataMemory RAM_Unit (  
    .clk(clk),  
    .MemWrite(write_mem),  
    .MemRead(read_mem),  
    .address(execution_res[8:0]),  
    .write_data(rs2_val),  
    .read_data(ram_data_read)  
);  
// Final Mux to choose if data comes from ALU or RAM  
mux2 WriteBack_Mux (  
    .reset(reset),  
    .d0(execution_res),  
    .d1(ram_data_read),  
    .sel(to_reg_mux),  
    .out(wb_result)  
);  
  
endmodule
```



*Datapath Diagram:*

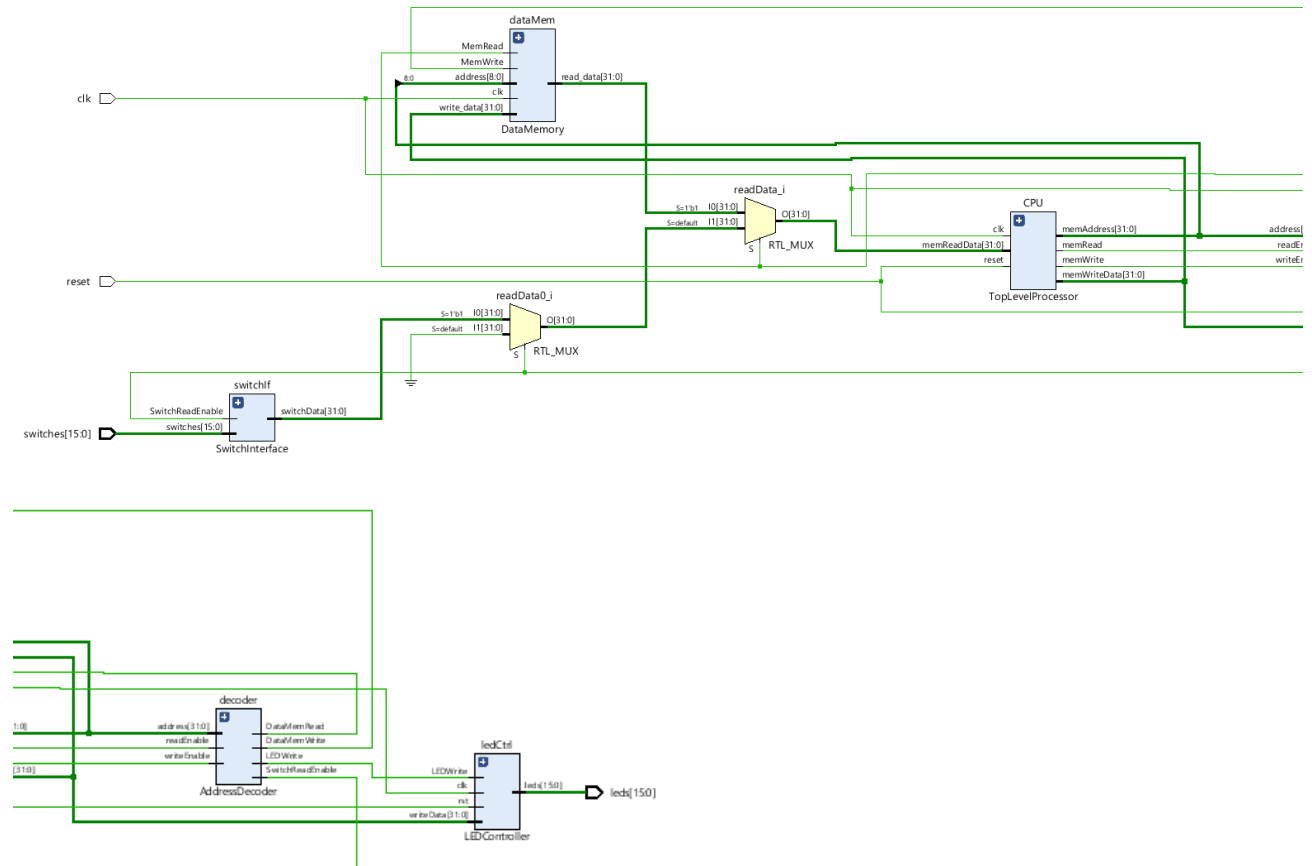




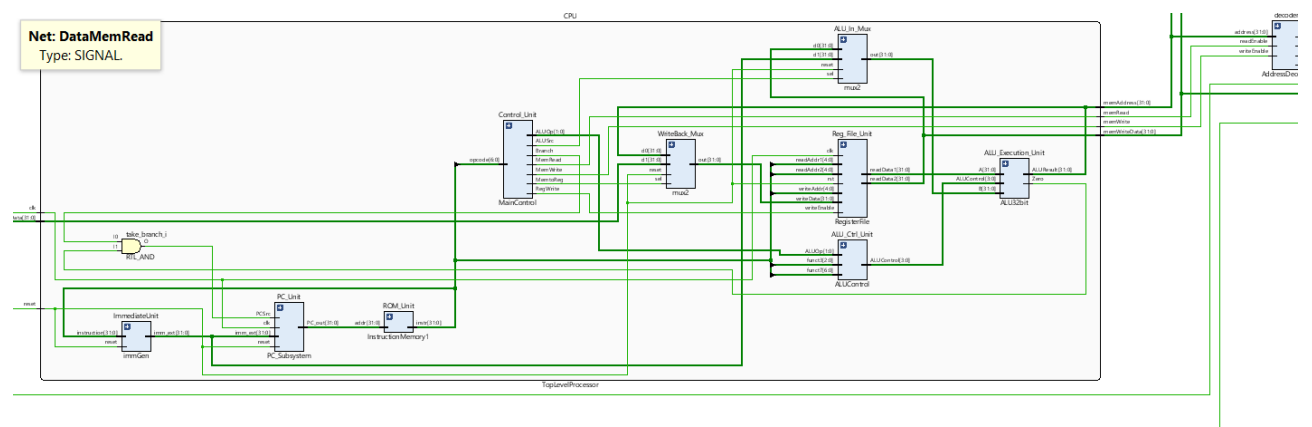
### RTL Schematic:

From left to right pictures are attached of whole top processor:

### TOP LEVEL PROCESSOR:



### CPU SCHEMATIC:



*Timing Analysis Report:***Design Timing Summary**

| Setup                        |    | Hold                         |    | Pulse Width                              |    |
|------------------------------|----|------------------------------|----|------------------------------------------|----|
| Worst Negative Slack (WNS):  | NA | Worst Hold Slack (WHS):      | NA | Worst Pulse Width Slack (WPWS):          | NA |
| Total Negative Slack (TNS):  | NA | Total Hold Slack (THS):      | NA | Total Pulse Width Negative Slack (TPWS): | NA |
| Number of Failing Endpoints: | NA | Number of Failing Endpoints: | NA | Number of Failing Endpoints:             | NA |
| Total Number of Endpoints:   | NA | Total Number of Endpoints:   | NA | Total Number of Endpoints:               | NA |

**All user specified timing constraints are met.**

**NOTE:** As the timing report shows all NA that means the test program is not running properly. After debugging and the code, im still unable to find the problem, hence the diagram and utilization report will be shown to you once the program runs perfectly. There is a small issue with *PC sub-system Module*, that will be fixed and results will be shared to the instructor.



## Lab 11: Design and FPGA Implementation of a Single-Cycle RISC-V Processor

### Points Distribution

| Task No.     | OE2 (Design) | OE5 (Report) | OE6 (Reflection) | AR7 |
|--------------|--------------|--------------|------------------|-----|
| Task 1       | /40          | -            | -                | /20 |
| Task 2       | /20          | /30          |                  |     |
| Viva         |              |              | /10              |     |
| Total Points | /80          |              |                  | /20 |
| CLO Mapped   | CLO 3        |              |                  |     |

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

| #   | Assessment Elements                            | Level 1: Unsatisfactory Points 0-1                                                                                                                                                         | Level 2: Developing Points 2                                                                                                                                                  | Level 3: Good Points 3                                                                                                                                     | Level 4: Exemplary Points 4                                                                                                                                                   |
|-----|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LR2 | Program/Code / Simulation Model/ Network Model | Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software. | Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software. | Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.        | Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.                      |
| LR5 | Results & Plots                                | Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.            | Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.                                        | All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.                                               | Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.                                |
| AR9 | Report                                         | All the in-lab tasks are not included in report and / or the report is submitted too late.                                                                                                 | Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.                  | Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely. | Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables. |